

1 Diffusion models

Diffusion models work by adding noise to destroy training data, then learning how to recover the original object. While the concept of using diffusion processes for generative modeling came earlier, [Ho et al. \(2020\)](#) proposed an approach that can generate photorealistic images. This note describes their model.

Define the following objects:

- $\mathbf{x}_{0:T} = (\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) \in (\mathbb{R}^d)^{T+1}$ is the trajectory of variables, where $\mathbf{x}_0$ is the original data and $\mathbf{x}_1, \dots, \mathbf{x}_T$ are latent variables.
- A **diffusion model** has the form

$$p_\theta(\mathbf{x}_0) := \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

i.e. the model's density on observed data is obtained by marginalizing the joint density over the path of the latent variables $\mathbf{x}_T, \dots, \mathbf{x}_1$.

- The **reverse process** is a Markov chain with Gaussian transitions

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)).$$

The notation refers to the normal distribution with the stated parameters evaluated at $\mathbf{x}_{t-1}$ and started from the standard Gaussian prior $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$, so that the joint factors as

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t). \quad (1)$$

Following the paper, the covariance is not learned but fixed to $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$, with either $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \tilde{\beta}_t$, where $\tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$.

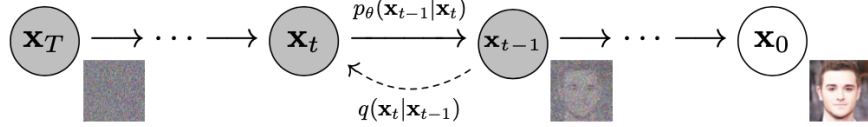
- The **forward process** is given by

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

where β_t is a variance schedule. Typically, we set it to increase with t , for example linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ across $T = 1000$. Using the

notation $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, we can write using standard Markov chain transition arguments the conditional density

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}).$$



Optimization is done via minimizing negative log-likelihood,

$$\begin{aligned} \mathbb{E}_{q(\mathbf{x}_0)}[-\log p_\theta(\mathbf{x}_0)] &= \mathbb{E}_{q(\mathbf{x}_0)} \left[-\log \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \right] \\ &= \mathbb{E}_{q(\mathbf{x}_0)} \left[-\log \int q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} d\mathbf{x}_{1:T} \right] \\ &= \mathbb{E}_{q(\mathbf{x}_0)} \left[-\log \mathbb{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \right] \right] \\ &\leq \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \right] \quad (\text{by Jensen's inequality}) \\ &= \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right] =: L. \end{aligned}$$

This can of course also be written in the form of a sum of KL divergences. Indeed, conditioning on $\mathbf{x}_0$ makes the forward posterior tractable,

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}), \quad \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t,$$

so that

$$L = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}_{L_0} \right].$$

With $\boldsymbol{\Sigma}_\theta = \sigma_t^2 \mathbf{I}$ fixed, each KL term is a plain quadratic, $L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 \right] + C$. Writing $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and parametrizing

the mean as

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \quad (2)$$

turns this into a noise-prediction problem,

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{(1 - \alpha_t)^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2 \right].$$

Ho et al. find that *dropping* the leading weight and sampling t uniformly gives better samples, yielding the simplified objective

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2,$$

which is what Algorithm 1 below minimizes. Algorithm 2 is then just ancestral sampling from p_θ with the mean (2).

The training algorithms works as follows. Let $\boldsymbol{\epsilon}_\theta$ be a neural network mapping the inputs to a target noise $\boldsymbol{\epsilon}$.

Algorithm 1 (Training).

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on
 $\quad \nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$
- 6: **until** converged

Algorithm 2 (Sampling).

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$

References

HO, J., A. JAIN, AND P. ABBEEL (2020): “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems*, vol. 33, 6840–6851.